



8. Conhecendo tuplas em Python

☒ Dúvidas?	☐
☒ Exercícios a entregar?	☒
 Módulo	1 - Primeiros passos com Python e Versionamento de código
 Tipo de aula	Teste Vídeo
 Links e mídias	Pasta "Aula 8"
 Última atualização	@17 de novembro de 2025 03:59
 Status	Done
 Data início	@09/11/2025
 Data conclusão	@09/11/2025 23:54
 Esforço	Médio

[Anotações](#)

[Vídeo 1 - Tuplas](#)

[Etapa 1: Criação e acesso aos dados](#)

[Exemplos de tuplas](#)

[Acesso direto](#)

[Tuplas aninhadas](#)

[Fatiamento](#)

[Etapa 2: Métodos da classe `tuple`](#)

`()`.count

`()`.index

len

[Exercícios](#)

[Arquivos](#)

Anotações

Vídeo 1 - Tuplas

Etapa 1: Criação e acesso aos dados

Tuplas são estruturas de dados muito parecidas com listas, porém são **imutáveis**. Tuplas também são mais restritas do que listas.

Para criar (declarar) tuplas, podemos usar o construtor `tuple` ou parênteses `( , )`, separando os valores dentro deles por vírgulas `( , )`.

Como os parênteses também são usados para determinar a precedência de operações, é prática recomendada **colocar uma vírgula após o último elemento** da tupla para que o interpretador não se confunda.

Exemplos de tuplas

```
frutas = ("laranja", "pera", "uva",)
```

```
letras = tuple("python")
```

```
numeros = tuple([1, 2, 3, 4])
```

```
pais = ("Brasil",)
```

Acesso direto

Assim como as listas, a tupla é uma sequência, e acessamos seus índices da mesma forma. Lembrando que o índice no Python começa de 0.

Assim como as listas, as tuplas também aceitam índices negativos, que funcionam da mesma forma.

```
frutas = ("maca", "laranja", "uva", "pera",)

frutas[0] # maca
frutas[2] # uva
frutas[-1] # pera
frutas[-3] # laranja
```

Tuplas aninhadas

Como as tuplas podem armazenar vários tipos de objetos, inclusive outras tuplas, podemos criar estruturas bidimensionais (tabelas) e acessar seus dados informando os índices de linha e coluna.

```
matriz = (
    (1, "a", 2), # isto é uma linha, com 3 colunas
    ("b", 3, 4),
    (6, 5, "c"),
) # esta matriz possui 3 linhas e 3 colunas, uma matriz 3 x 3

matriz[0] # [1, "a", 2]
matriz[0][0] # 1
matriz[0][-1] # 2
matriz[-1][-1] # c
```

Fatiamento

Além de acessar elementos diretamente, podemos extrair um conjunto de valores de uma sequência. Para isso, basta passar o índice inicial e/ou final para acessar o conjunto. Podemos, também, informar quantas posições o cursor deve "pular" no acesso.

```
tupla = ("p", "y", "t", "h", "o", "n",)
```

```
print(tupla[2:]) # ["t", "h", "o", "n"], do índice 2 até o final
```

```
print(tupla[:2]) # ["p", "y"], até o índice 1
```

```
print(tupla[1:3]) # ["y", "t"], do índice 1 ao 2
```

```
print(tupla[0:3:2]) # ["p", "t"], do índice 0 até o 2, contando de 2 em 2. O índice 0, aqui, poderia ser omitido.
```

```
print(tupla[:]) # ["p", "y", "t", "h", "o", "n"], todos os índices
```

```
print(tupla[::-1]) # ["n", "o", "h", "t", "y", "p"], todos os índices, de trás para frente
```

Assim como em listas, podemos iterar uma tupla com for e usar a função

`enumerate` .

Etapa 2: Métodos da classe `tuple`

Há menos métodos para tuplas do que para listas, vejamos os disponíveis abaixo.

`()`.count

Conta a ocorrência dos elementos em uma tupla.

```
cores = ("vermelho", "azul", "verde", "azul", "amarelo", "verde",)
```

```
print(cores.count("vermelho")) # 1
```

```
print(cores.count("azul")) # 2
```

```
print(cores.count("verde")) # 2
```

```
print(cores.count("amarelo")) # 1
```

`()`.index

Exibe o número do índice do parâmetro colocado entre os parênteses.

```
linguagens = ("python", "js", "c", "java", "csharp",)
```

```
print(linguagens.index("java")) # 3
```

```
print(linguagens.index("python")) # 0
```

Lembrando que é exibida a primeira ocorrência do termo entre parênteses, então se há duas ocorrências de um termo, é exibido o índice da primeira. Combinando este método com `count`, podemos alterar todas as ocorrências de um termo, até que o `count` seja `0`.

`len`

Esta função que acompanha a instalação exibe o comprimento de um elemento da tupla ou de uma tupla em si.

```
linguagens = ["python", "js", "c", "java", "csharp"]  
  
print(len(linguagens)) # 5  
print(len(linguagens[3])) # 4
```

Exercícios

75% ⇒ 3/4 corretas

Arquivos

Pasta "Aula 8"